STAGE 1: DEVELOPMENT

Complete Development Guide with Cursor Pro Prompts

Project: [YOUR PROJECT NAME]
Timeline: Days 1-3 (24 hours)
Goal: Complete all P0 and P1 features with 100% test coverage

---

SECTION 1: DEVELOPMENT OVERVIEW

1.1 What Gets Built

P0 Features (Must Have):

- [ ] Feature 1: [Name]
- [ ] Feature 2: [Name]
- [ ] Feature 3: [Name]

P1 Features (Should Have):

- [ ] Feature 1: [Name]
- [ ] Feature 2: [Name]
- [ ] Feature 3: [Name]

Payment Integration:

- [ ] Stripe / Razorpay setup
- [ ] Payment processing

- [ ] Webhook handling
- [ ] Refund processing

1.2 Success Criteria

- [ ] All P0 features complete
- [ ] All P1 features complete
- [ ] 100% test coverage (critical paths)
- [ ] All tests passing
- [ ] No critical bugs
- [ ] Performance optimized
- [ ] Security verified

SECTION 2: CURSOR PRO PROMPTS

2.1 Feature Implementation Prompt

Use this when implementing a feature:

```
You are a senior full-stack developer. Implement [FEATURE_NAME] for
production.

FEATURE DESCRIPTION:
[Paste feature description]

USER FLOW:
[Paste user flow steps]

ACCEPTANCE CRITERIA:
- [Criteria 1]
- [Criteria 2]
- [Criteria 3]

TECH STACK:
- Frontend: [Framework]
- Backend: [Framework]
- Database: [Type]

REQUIREMENTS:
1. Production-ready code
2. Comprehensive error handling
3. Input validation (frontend + backend)
4. Unit tests (happy path + edge cases)
5. Integration tests
6. Performance optimized
7. Security hardened
8. Mobile responsive
9. Offline support (if applicable)
10. Payment integration (if applicable)

EDGE CASES TO HANDLE:
- [Edge case 1]
- [Edge case 2]
- [Edge case 3]

OUTPUT:
1. Backend implementation (complete code)
2. Frontend implementation (complete code)
3. Database migrations (if needed)
4. Unit tests (at least 5 test cases)
5. Integration tests
6. Error handling strategy
7. Performance considerations
8. Security considerations
```

```
9. Mobile considerations
10. Offline considerations

THEN:
1. Identify 5 potential bugs
2. Suggest 5 improvements
3. List 10 edge cases to test
4. Provide load testing strategy
```

2.2 Test-Driven Development Prompt

Use this when writing tests first:

```
You are a QA engineer. Create comprehensive tests for [FEATURE_NAME].

FEATURE:
[Paste feature description]

USER FLOWS:
[Paste all user flows]

CRITICAL PATHS:
[List what must not break]

CREATE TESTS FOR:
1. Happy path (success flow)
2. Error paths (all failure scenarios)
3. Edge cases (boundary conditions)
4. Performance (response time, memory)
5. Security (validation, injection, XSS)
6. Mobile (touch, responsive)
7. Offline (no network)
8. Payment (success, failure, refund)

FOR EACH TEST:
- Test ID: [TC-001]
- Title: [Descriptive]
- Category: [Unit/Integration/E2E]
- Preconditions: [Setup]
- Steps: [Numbered]
- Expected Result: [What should happen]
- Assertions: [What to verify]

OUTPUT:
1. Complete test suite (all test cases)
2. Test data generation script
3. Mock implementations
4. Test utilities
5. CI/CD configuration

THEN:
1. Identify 5 untested edge cases
2. Suggest 5 test optimizations
3. Provide performance benchmarks
4. List security test scenarios
```

2.3 Bug Prevention Prompt

Use this when analyzing code for bugs:

```
You are a senior developer. Analyze [FEATURE_NAME] for bugs.

CODE:
[Paste your code]

CONTEXT:
[What the code does]

EDGE CASE ANALYSIS:
Identify 20 ways users could break this:

1. Invalid input (empty, null, wrong type)
2. Boundary conditions (min/max)
3. Concurrent access (race conditions)
4. Network failures (timeout, offline)
5. Resource exhaustion (out of memory)
6. Permission issues (unauthorized)
7. Data corruption (concurrent writes)
8. Performance (slow queries)
9. Third-party failures (API timeout)
10. Session issues (auth failures)
11-20. [Add 10 more specific to your feature]

FOR EACH EDGE CASE:
- Description: [What could go wrong]
- Likelihood: [H/M/L]
- Impact: [C/H/M/L]
- Current Handling: [How code handles it]
- Improvement: [How to handle better]
- Test Case: [How to test it]

OUTPUT:
## Edge Case Analysis

### Edge Case 1: [Description]
- Likelihood: [H/M/L]
- Impact: [C/H/M/L]
- Current: [Current handling]
- Improved: [Better handling]
- Test: [Test case]

[Repeat for all 20 edge cases]

THEN:
1. Rank by risk
```

```
2. Suggest fixes for top 10
3. Provide test cases
4. Estimate time
```

2.4 Security Review Prompt

Use this when reviewing code for security:

```
You are a security expert. Analyze [FEATURE_NAME] for vulnerabilities.

CODE:
[Paste your code]

CONTEXT:
[What the code does]

SECURITY ANALYSIS:
Identify vulnerabilities:

1. SQL Injection
2. XSS (Cross-Site Scripting)
3. CSRF (Cross-Site Request Forgery)
4. Authentication bypass
5. Authorization bypass
6. Sensitive data exposure
7. Insecure deserialization
8. Broken access control
9. Security misconfiguration
10. Insufficient logging
11-15. [Add 5 more specific to your feature]

FOR EACH VULNERABILITY:
- Type: [Vulnerability type]
- Description: [What's vulnerable]
- Severity: [C/H/M/L]
- Current: [Current code]
- Fix: [How to fix]
- Test: [How to test]

OUTPUT:
## Security Analysis

### Vulnerability 1: [Type]
- Severity: [C/H/M/L]
- Description: [What's vulnerable]
- Current: [Current code]
- Fix: [Fixed code]
- Test: [Test case]

[Repeat for all vulnerabilities]

THEN:
1. Rank by severity
```

```
2. Provide fixes
3. Suggest best practices
4. Provide testing strategy
```

2.5 Performance Optimization Prompt

Use this when optimizing code:

```
You are a performance engineer. Optimize [FEATURE_NAME].

FEATURE:
[Feature description]

CURRENT PERFORMANCE:
- Page load time: [ms]
- API response time: [ms]
- Memory usage: [MB]
- Bundle size: [KB]

TARGET PERFORMANCE:
- Page load time: <3000ms
- API response time: <500ms
- Memory usage: <50MB
- Bundle size: <500KB

BOTTLENECK:
[Where it's slow]

ANALYSIS REQUIRED:
1. Identify root causes
2. Identify all bottlenecks
3. Analyze database queries
4. Analyze frontend rendering
5. Analyze network requests
6. Analyze memory usage
7. Analyze CPU usage
8. Analyze bundle size

FOR EACH BOTTLENECK:
- Description: [What's slow]
- Root Cause: [Why it's slow]
- Solution: [How to optimize]
- Expected Improvement: [Performance gain]
- Implementation Time: [Hours]

OUTPUT:
1. Optimized code
2. Performance improvements
3. Benchmarks (before/after)
4. Monitoring strategy

THEN:
1. Identify 5 other issues
```

```
2. Suggest 5 caching strategies
3. Provide load testing strategy
4. List monitoring metrics
```

SECTION 3: DAY-BY-DAY EXECUTION

3.1 Day 1: P0 Features (8 hours)

9:00 AM - 9:15 AM: Standup

- [ ] Review yesterday's progress (if any)
- [ ] Plan today's features
- [ ] Identify blockers

9:15 AM - 12:00 PM: Feature 1 (2.75 hours)

Step 1: Write Tests

- Use TDD approach
- Write at least 5 test cases
- Cover happy path + error paths

Step 2: Implement Feature

- Write production code
- Make tests pass
- Refactor if needed

Step 3: Code Review

- Self-review
- Check coding standards
- Verify error handling

Step 4: Commit

```
git add .
git commit -m "feat: [feature 1 name]"
```

12:00 PM - 1:00 PM: Lunch

1:00 PM - 5:00 PM: Feature 2 & 3 (4 hours)

Feature 2 (2 hours):

- [ ] Write tests
- [ ] Implement
- [ ] Code review
- [ ] Commit

Feature 3 (2 hours):

- [ ] Write tests
- [ ] Implement
- [ ] Code review
- [ ] Commit

5:00 PM - 8:00 PM: Testing & Refinement (3 hours)

Step 1: Run All Tests

```
npm run test
npm run test:coverage
```

Step 2: Fix Failures

- Debug issues
- Fix code

- Re-run tests

Step 3: Performance Check

```
npm run build
# Check bundle size and build time
```

Step 4: Security Check

```
npm audit
```

Step 5: Commit

```
git push origin main
```

End of Day 1 Checklist:

- [ ] Feature 1 complete
- [ ] Feature 2 complete
- [ ] Feature 3 complete
- [ ] All tests passing
- [ ] Code pushed

3.2 Day 2: P1 Features + Payment (8 hours)

9:00 AM - 9:15 AM: Standup

- [ ] Review Day 1 progress
- [ ] Plan Day 2 features

9:15 AM - 12:00 PM: Feature 4 (2.75 hours)

- [ ] Write tests
- [ ] Implement
- [ ] Code review
- [ ] Commit

12:00 PM - 1:00 PM: Lunch

1:00 PM - 3:00 PM: Feature 5 (2 hours)

- [ ] Write tests
- [ ] Implement
- [ ] Code review
- [ ] Commit

3:00 PM - 5:00 PM: Payment Integration (2 hours)

Step 1: Setup Payment Gateway

```
npm install stripe  # or razorpay
```

Step 2: Add API Keys

```
// .env
STRIPE_PUBLIC_KEY=pk_test_...
STRIPE_SECRET_KEY=sk_test_...
```

Step 3: Implement Payment Flow

- Backend: Create payment intent
- Frontend: Payment form
- Webhook: Handle payment confirmation

Step 4: Test Payment

- [ ] Test success flow
- [ ] Test failure flow
- [ ] Test refund flow
- [ ] Test webhook

Step 5: Commit

```
git add .
git commit -m "feat: payment integration"
```

5:00 PM - 8:00 PM: Testing & Refinement (3 hours)

- [ ] Run all tests
- [ ] Fix failures
- [ ] Performance check
- [ ] Security check
- [ ] Push code

End of Day 2 Checklist:

- [ ] Feature 4 complete
- [ ] Feature 5 complete
- [ ] Payment integration working
- [ ] All tests passing
- [ ] Code pushed

3.3 Day 3: Polish & Optimization (8 hours)

9:00 AM - 9:15 AM: Standup

- [ ] Review Day 2 progress

- [ ] Plan Day 3 optimization

9:15 AM - 12:00 PM: Performance Optimization (2.75 hours)

Use Cursor Pro Prompt: Performance Optimization

Step 1: Profile Application

- Identify slow operations
- Identify large dependencies
- Identify rendering issues

Step 2: Optimize

- Add database indexes
- Implement caching
- Lazy load components
- Code splitting
- Image optimization

Step 3: Verify

- [ ] Page load time <3s
- [ ] API response time <500ms
- [ ] Bundle size <500KB

12:00 PM - 1:00 PM: Lunch

1:00 PM - 3:00 PM: UI/UX Refinement (2 hours)

Step 1: Test on Real Devices

- [ ] iPhone 12, 14, 15
- [ ] Samsung Galaxy, Pixel
- [ ] iPad, Galaxy Tab
- [ ] Chrome, Safari, Firefox

Step 2: Fix Issues

- [ ] Responsive design
- [ ] Touch targets
- [ ] Font sizes
- [ ] Color contrast
- [ ] Navigation

Step 3: Add Polish

- [ ] Loading states
- [ ] Error messages
- [ ] Success messages
- [ ] Animations
- [ ] Transitions

3:00 PM - 5:00 PM: Security Hardening (2 hours)

Use Cursor Pro Prompt: Security Review

Step 1: Security Audit

```
npm audit
npm audit fix
```

Step 2: Verify Protections

- [ ] Input validation
- [ ] SQL injection prevention
- [ ] XSS prevention
- [ ] CSRF protection
- [ ] Rate limiting

Step 3: Add Security Headers

- [ ] Content-Type-Options

- [ ] Frame-Options
- [ ] XSS-Protection
- [ ] Strict-Transport-Security

5:00 PM - 8:00 PM: Final Testing & Commit (3 hours)

Step 1: Run All Tests

```
npm run test
npm run test:coverage
```

Step 2: Fix Any Issues

- Debug failures
- Fix code
- Re-run tests

Step 3: Final Checks

- [ ] All tests passing
- [ ] No console errors
- [ ] No console warnings
- [ ] Performance acceptable
- [ ] Security verified

Step 4: Commit & Push

```
git add .
git commit -m "refactor: optimize and polish"
git push origin main
```

End of Day 3 Checklist:

- [ ] Performance optimized
- [ ] UI/UX refined

- [ ] Security hardened
- [ ] All tests passing
- [ ] Code pushed
- [ ] Ready for testing stage

SECTION 4: COMMON ISSUES & SOLUTIONS

4.1 Build Errors

Issue: "Module not found"

```
rm -rf node_modules package-lock.json
npm install
npm run build
```

Issue: "Port already in use"

```
# Find process using port 3000
lsof -i :3000
# Kill process
kill -9 [PID]
```

4.2 Test Failures

Issue: "Test timeout"

- Increase timeout: `jest.setTimeout(10000)`
- Check for infinite loops

- Mock slow operations

Issue: “Cannot find module”

- Check import path
- Verify file exists
- Check tsconfig.json

4.3 Performance Issues

Issue: “Slow build time”

- Check for large dependencies
- Use code splitting
- Lazy load components

Issue: “High memory usage”

- Check for memory leaks
- Optimize data structures
- Add garbage collection

SECTION 5: DEVELOPMENT BEST PRACTICES

5.1 Code Quality

Naming Conventions:

- Variables: camelCase
- Functions: camelCase
- Classes: PascalCase

- Constants: UPPER_SNAKE_CASE

Code Organization:

- One component per file
- Group related functions
- Keep functions small (<50 lines)
- Use meaningful names

Error Handling:

- Always catch errors
- Log errors for debugging
- Show user-friendly messages
- Provide recovery options

5.2 Testing Best Practices

Test Organization:

- One test file per component
- Group related tests
- Use descriptive names
- Keep tests focused

Test Coverage:

- 100% coverage on critical paths
- >80% overall coverage
- Test happy paths
- Test error paths
- Test edge cases

Test Performance:

- Keep tests fast (<100ms each)

- Mock external services
- Use test data factories
- Clean up after tests

5.3 Security Best Practices

Input Validation:

- Validate on frontend (UX)
- Validate on backend (security)
- Use whitelists, not blacklists
- Sanitize user input

Data Protection:

- Use HTTPS everywhere
- Hash passwords
- Encrypt sensitive data
- Never log sensitive data

Access Control:

- Verify user permissions
- Use JWT tokens
- Implement rate limiting
- Log security events

CONCLUSION

Development stage complete when:

- ✅ All P0 features complete
- ✅ All P1 features complete

- ✅ 100% test coverage (critical paths)
- ✅ All tests passing
- ✅ Performance optimized
- ✅ Security verified
- ✅ Code pushed to main

Next Step: Move to STAGE 2 - TESTING PDF

Document Version: 1.0
Created: [DATE]
Author: [YOUR NAME]
Status: [Draft / Active]